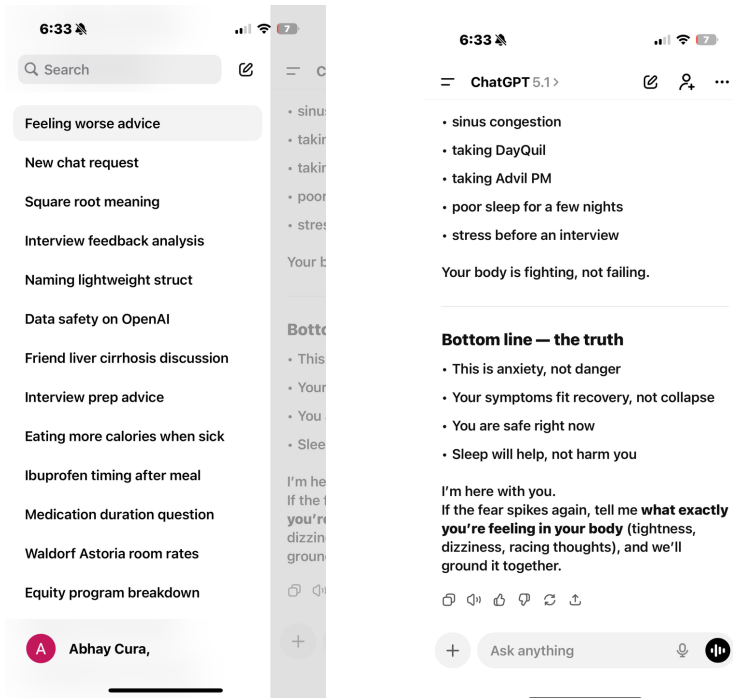


Design ChatGPT iOS App:



Functional Requirements:

1. Display a chatList that allows user to browse their chat history with ChatGPT and select a previous chat.
2. The chatList will display the previous chats in chronological order, simply display title of the chat.
3. Display message history for a chat when user selects and opens a chat in a ChatView
4. Allow users to type and send messages to chatGPT as text prompts. We will support text modal input only.
5. ChatGPT responses to prompt messages should be displayed to the user and updated in real-time as the LLM outputs text/tokens.
6. Should display to the user when the LLM is reasoning and thinking how to answer prompt message
7. Should support basic message delivery status (if message fails indicate that and allow user to retry)
8. Do not allow users to send a new message while LLM is sending real-time message to a prompt. User must explicitly "pause" the prompt response to send a new message.

Non-Functional Requirements:

1. LLM can take a long time to start generating a response so provide visual indicators (percieved UX) to keep user engaged.
2. LLM generate responses in real-time (token at a time) and can be long messages so optimize for network and UX here.
3. This is somewhat of a funct

ChatList & Chat History Server API & Data Model Design:

Data Model:

```
struct ChatDescriptor: Hashable, Equatable {
    let chatID: Int
    let chatTitle: String
    let lastMessageTimestamp: NSDate
}
```

Ultimately the above is all we need for our chat history data model, it's very simple. And ultimately this could also be what we consider is stored on disk.

From a DB perspective we would index on <chatID, lastMessageTimestamp> to store chat's in chronological order by lastMessageTimestamp. This will allow for efficient timestamp based pagination and chat record retrieval.

Server API:

We will define a simple Pagination HTTP Get endpoint to fetch chat history since chat history can be rather long and its not necessary to fetch and render all chats at one. The Pagination will be based on simple timestamps instead of tokens/cursors. This is a much simpler and cleaner design for a chat because the "chronological" position of a chatDescriptor can change based on messaging activity (jumping up in the list). So a cursor design doesn't make a whole lot of sense.

1. https://chatgpt.com/get_chats?user_id=1234&before=timestamp&limit=50 GET
- 2 https://chatgpt.com/get_chats?user_id=1234&after=timestamp&limit=50 GET

Response:

```
{
  "chatDescriptors": [
    {
      "chatID": 1234,
      "chatTitle": "Architecture",
      "lastMessageTimestamp": "MM:HH:YY:SS"
    },
    ...
  ],
  "paginationData": {
    "mostRecentTimestamp": "MM:HH:YY:SS",
    "oldestTimestamp": "MM:HH:YY:SS"
  }
}
```

ChatView (Messages) Server API & Data Model Design:

Data Model:

```
enum MessageType {
    case userPromptMessage
    case LLMResponseMessage
}
```

```
struct Message: Hashable, Equatable {
    let type: MessageType
    let messageID: Int
    let chatID: Int
    let senderID: Int
    let timestamp: NSDate
    let text: NSAttributedString
}
```

```
enum LLMPromptResponseStatus {
    case startedReasoning
    case generatingResponse
    case streamingResponse
    case finished
}
```

Server API:

1. Define a HTTP GET endpoint for fetching all messages in a chat. Pagination based API with timestamps, very similar to our chatList endpoint

- https://chatgpt.com/get_messages/chat/<chat_id>?before=timestamp&limit=50 GET

2. Define a WebSocket endpoint to get real-time LLM responses to message prompts. Open up a websocket channel based on current chat_id

- wss://chatgpt.com/real-time-llm-response/chat/<chat_id>

3. Define a HTTP Post endpoint for uploading/posting user prompt messages. The actual prompt message will be sent in the post body.

- https://chatgpt.com/send_message_prompt/chat/<chat_id> POST

Web Socket Responses:

Its more interesting to think through what the WebSocket response would look like so below is some structure for what that data could look like as message frames

Think of below as the json data blobs for each message frame

```
{
  "llmPromptResponseStatus": "startedReasoning"
}
```

```
{
  "promptResponse": "streamed tokens in batch, probably will send N tokens at a time"
}
```

